

Atividade de Reforço

Criando Banco, Tabelas e Populando com Dados

Contexto: você foi contratado por uma escola para organizar o cadastro de cursos e alunos.

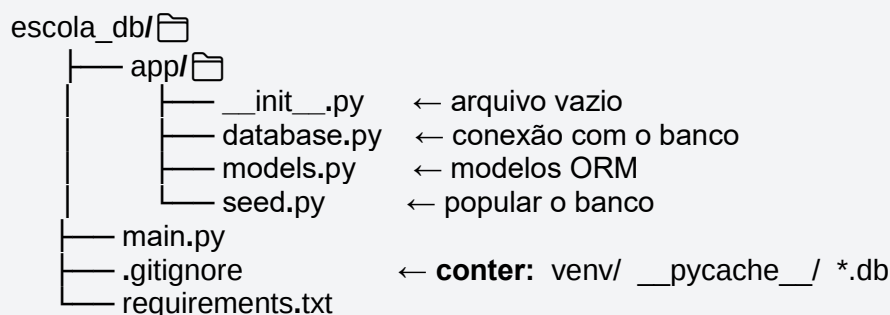
Sua tarefa é criar o banco de dados do sistema usando Python e SQLAlchemy.

Banco de dados: escola.db

Tabelas: cursos e alunos

Parte 1 — Estrutura do projeto

Criar a pasta **escola_db** e dentro dela montar a estrutura abaixo. Depois instalar as dependências.



Após finalizar a estrutura crie e ative o ambiente virtual:

```
python -m venv venv
source venv\Scripts\activate
```

Instale a dependência:

```
pip install sqlalchemy
pip freeze > requirements.txt
```

Parte 2 — Configurar o database.py

Complete o arquivo **app/database.py** com as lacunas:

```
from sqlalchemy import create_engine
from sqlalchemy.orm import ~~, DeclarativeBase
                        # ↑ qual import?

# URL do banco SQLite - vai criar o arquivo escola.db
DB_URL = '~~:////escola.db'
                # ↑ qual protocolo?

engine = ~( DB_URL, echo=True, connect_args={'check_same_thread':
False}) # ↑ qual função cria o engine?

SessionLocal = ~(bind=engine, autocommit=False, autoflush=False)
                # ↑ qual função cria a fábrica de sessões?
```

```
class Base(DeclarativeBase):
```

```
# ↑ o corpo da classe fica vazio com qual palavra?
```

Parte 3 — Criar os modelos ORM

Complete os dois modelos abaixo em `app/models.py`:

Tabela	Campo	Tipo	Observação
cursos	id	Integer	Chave primária, autoincremento
cursos	nome	String(100)	Obrigatório
cursos	duracao	Integer	Duração em horas, obrigatório
cursos	ativo	Boolean	Padrão True
alunos	id	Integer	Chave primária, autoincremento
alunos	nome	String(150)	Obrigatório
alunos	email	String(100)	Obrigatório e único
alunos	matricula	String(20)	Obrigatório e único
alunos	ativo	Boolean	Padrão True

```
from sqlalchemy import Column, Integer, String, Boolean
from app.database import Base
```

```
class Curso(Base):
    __tablename__ = ' '
    # ↑ O nome da classe será o nome da tabela
    id = Column(Integer, primary_key=True, =True)
    # Toda ↑ chave primaria de ser autoincremento
    nome = Column(String( ), nullable= )
    # ↑ tamanho do texto ↑ campo obrigatório
    duracao = Column( , nullable=False)
    # ↑ tipo inteiro
    ativo = Column(Boolean, default= )
    # ↑ valor padrão

    def __repr__(self):
        return f'<Curso id={self. } nome={self.nome}>'
        # ↑ campo de ID
```

```
class Aluno(Base):
    __tablename__ = ' '
    # ↑ O nome da classe será o nome da tabela

    id = Column(Integer, primary_key=True, autoincrement=True)
    nome = Column(String(150), nullable= )
    # ↑ campo obrigatório
```

```

email = Column(String(100), nullable=False, unique=True)
# ↑ campo único
matricula = Column(String(50), nullable=False, unique=True)
# ↑ tamanho do texto recebido pelo banco
ativo = Column(Boolean, default=True)
# ↑ tipo a ser recebido

def __repr__(self):
    return f'<Aluno id={self.id} nome={self.nome>'
# ↑ campo do nome

```

Parte 4 — Criar as tabelas no banco

Complete o `main.py` para criar as tabelas no banco:

```

from app.database import engine, Base
# ↑ o que mais é preciso importar?

from app import models
# ↑ qual módulo importar para registrar os modelos?

# Criar todas as tabelas que não existem
Base.metadata.create_all(bind=engine)
# ↑ o objeto      ↑ qual método?

print('Tabelas criadas!')

```

Antes de prosseguir teste, executando o arquivo: `python main.py`
 No terminal (echo=True): deve aparecer o SQL de CREATE TABLE
 No SQLite Viewer: abrir o arquivo escola.db
 Verificar: as tabelas 'cursos' e 'alunos' aparecem
 Verificar: clicar em cada tabela e ver as colunas

Parte 5 — Popular o banco com seed.py

Complete o arquivo `app/seed.py` para inserir os dados iniciais:

```

from app.database import Session
# ↑ o que importar para criar sessão?

from app.models import Curso, Aluno
# ↑ e qual outro modelo?

def popular_banco():
    db = Session() # abrir sessão
    # ↑ qual função?

    try:
        if db.query(Curso).count() > 0:
            print('Banco já populado. Pulando...')
            return

        # Inserir cursos
        db.add([
            # ↑ qual método para lista?

```

```

    ██████████ (nome='Desenvolvimento de Sistemas', duracao=1200),
    # ↑ qual classe usar?
    # Adicione um segundo item na tabela de curso,
    # Adicione um terceiro item na tabela de curso,
    ])

    # Inserir alunos
    db.add_all([
        ██████████ (nome='Lucas Mendes', email='lucas@senai.com',
matricula='2025001'),
    #     ↑ qual classe usar?
    # Adicione um segundo item na tabela de aluno,
    # Adicione um terceiro item na tabela de aluno,
    # Adicione um quarto item na tabela de aluno,
    ])

    db.██████████() # confirmar no banco
    # ↑ qual método?
    print('Banco populado com sucesso!')

except Exception as e:
    db.██████████() # desfazer se der erro
    # ↑ qual método?
    print(f'Erro: {e}')
finally:
    db.██████████() # sempre fechar
    # ↑ qual método?

if __name__ == '__main__':
    popular_banco()

```

Atualizar o main.py para chamar o seed:

Copie e cole no main.py

```

from app.seed import popular_banco
Base.metadata.create_all(bind=engine)
popular_banco()

```

Teste final, execute o arquivo: python main.py

Terminal: 'Banco populado com sucesso!'

SQLite Viewer: tabela cursos com 3 linhas, tabela alunos com 4 linhas

Executar novamente: 'Banco já populado. Pulando...'

Verificação Final

Antes de entregar, marque cada item como concluído:

- () Pasta escola_db criada com a estrutura correta
- () requirements.txt contém sqlalchemy
- () .gitignore contém venv/, __pycache__/ e *.db
- () database.py com DB_URL sqlite, engine, SessionLocal e Base
- () Modelo Curso com 4 campos (id, nome, duracao, ativo)
- () Modelo Aluno com 5 campos (id, nome, email, matricula, ativo)
- () email e matricula com unique=True no Aluno
- () python main.py cria as tabelas sem erro
- () SQLite Viewer mostra as 2 tabelas com as colunas corretas
- () Tabela cursos com 3 linhas no SQLite Viewer
- () Tabela alunos com 4 linhas no SQLite Viewer
- () Executar python main.py novamente: 'Banco já populado'
- () Arquivo escola.db NÃO aparece no GitHub Desktop (está no .gitignore)

Entrega:

Link do repositório pelo Google Classroom